

Data Dynamics - Interview Prep

React Native Offline-First Forms App | Structured Question & Answer Guide

Part 1: Project Introduction

Q1. Tell me about yourself (project portion).

In my current role, I work as a React Native developer on Data Dynamics, an offline-first mobile forms application used by field technicians to fill out dynamic, configurable inspection/service forms - even without network connectivity. The forms are schema-driven, so the same engine renders different forms with conditional logic like show/hide fields, dynamic validation, and required rules, without any code changes per form. I've worked on [pick: the offline sync/outbox queue using Realm / the annotation and PDF markup module using react-native-skia / the conditional logic engine], making sure data entered offline is never lost and reliably syncs once the device reconnects.

Q2. Explain Data Dynamics in 30 seconds (elevator pitch).

Data Dynamics is a React Native offline-first mobile forms application used by field workers to fill out dynamic, configurable forms - things like inspection checklists, service reports, etc. - even in areas with poor or no network connectivity. The forms aren't hardcoded; they're driven by a JSON schema that defines pages, sections, questions, validations and conditional logic, so the same app can render completely different forms without a code change. Users can attach photos, annotate/markup images and PDFs, capture signatures, and submit data that syncs to the backend via GraphQL/AWS AppSync once connectivity is restored.

Q3. What problem does it solve?

- Field technicians often work in locations with weak/no internet (basements, remote sites). They still need to fill out and submit forms.
- Forms change frequently per customer/project (different fields, different business rules) - rebuilding the app for every form variant isn't scalable.
- Forms often require visual evidence - photos with annotations/markup, signatures, PDF markups - not just text answers.

Part 2: Architecture

Q4. How does the dynamic form engine work?

Forms are rendered from JSON (pages -> sections -> questions). A `DataDynamicContextWrapper` (Context API) holds `formAnswers`, visibility/readonly state, and runs a **conditional logic engine** (`processConditionalLogic`, `processPageSectionVisibility`) that computes hidden/required/readonly fields reactively as the user types, including page/section-level visibility, not just field-level.

Q5. How did you implement offline-first storage?

Used **Realm** as the local database (`CachedForm`, `CachedSubmission`, `CachedDraft`, `CachedProjectForm`, `CachedUser` schemas). Forms, drafts, and submissions are cached locally so the app is fully usable offline.

Q6. How does the sync / outbox pattern work?

Submissions created offline are queued with a `status: "pending" | "synced"` flag. An outbox screen shows pending items; a sync manager retries uploads (including attachments) once network returns, then flips status to synced.

Q7. What does the networking layer look like?

`apolloClientService` (Apollo Client + AWS AppSync) handles GraphQL queries/mutations, plus a REST layer (`ApiService`, `axiosInstance`) for auxiliary calls like file upload/download. `NetworkInfo/networkHoc` (HOC) detects connectivity and gates sync behavior.

Q8. How do attachments and annotation work?

Users capture images (camera / image picker), and there's a full annotation module - `AnnotationCanvas`, `AnnotationEditor`, `AnnotationToolbar`, undo/redo history (`annotationHistory`), geometry helpers, serialization to persist markup - built with **react-native-skia** for performant freehand drawing, plus a **PDF annotation editor** (`react-native-pdf`, `pdf-lib`) so users can mark up PDF documents (not just images).

Q9. How is the app tested?

Jest + React Native Testing Library - dedicated test suites for annotation geometry, gestures, serialization, outbox, dropdown services, form filters, etc.

Part 3: Real Problems Faced and How They Were Solved

Pick 2-3 of these and be ready to go deep - interviewers love when you can explain the why, not just the what.

Q10. Tell me about a challenge with conditional logic / dynamic rendering.

Problem: Forms have cross-field rules (show/hide questions, pages, sections based on other answers; mark fields required/readonly dynamically) and naive re-computation on every keystroke caused lag and bugs where hidden fields still held stale values.

Fix: Centralized the logic into a pure "conditional logic engine" that takes current answers + rule config and derives hidden/required/readonly sets in one pass, memoized with `useCallback/useEffect`, and cleared/reset values for fields that become hidden so stale data isn't submitted.

Q11. How did you handle offline sync, conflicts and duplicate submissions?

Problem: User fills a form offline, app crashes or network flaps mid-upload - risk of duplicate submissions or lost data.

Fix: Every submission gets a client-generated UUID (`submissionId`) before it ever touches the network, stored in Realm with a `pending/synced` status. The outbox/sync manager processes a queue sequentially (not in parallel) so one failure doesn't corrupt others, and retries are idempotent because the server checks the same UUID.

Q12. How did you deal with large image uploads over unreliable networks?

Problem: Uploading several high-res photos per form on a flaky connection caused failures and poor UX.

Fix: Images are compressed/resized (`react-native-image-resizer`) before upload, uploads are queued one at a time with retry/backoff, and upload state is tracked separately from form status so a form can be "submitted" locally while attachments sync in the background.

Q13. How did you solve annotation / drawing performance issues?

Problem: Freehand drawing with React Native's default Canvas/SVG approach was laggy on lower-end devices, especially with undo/redo and many strokes.

Fix: Moved rendering to `react-native-skia` for GPU-accelerated drawing, kept stroke data in a serializable format (vector points, not bitmaps) so annotations could be undone/redone and re-rendered at any resolution, and persisted/restored them via a dedicated serializer.

Q14. How do you test an app with heavy native modules (Realm, camera, PDF, Skia)?

Problem: These native modules don't run in Jest's Node environment.

Fix: Used manual mocks (e.g. `react-native-image-crop-picker.js`) and dependency isolation so business logic (geometry, services, reducers/context) could be unit-tested without needing a real device/simulator.

Part 4: Your Personal Contribution

Q15. What exactly did you work on in this project?

Be ready with specifics: "I worked primarily on the Annotation module / offline sync / conditional logic / form renderer..." - pick whatever you actually built, and have 1 concrete bug or edge case you fixed, since interviewers often ask "tell me about a bug you fixed."

Part 5: Likely Follow-up Questions

Q16. How does Realm compare to AsyncStorage/SQLite - why did you choose it?

Realm gives object-based queries, reactive listeners, and better performance for structured relational-ish data vs key-value AsyncStorage.

Q17. How do you handle merge conflicts if the same form is edited on two devices?

Be honest about your actual approach - e.g., last-write-wins, or each submission is independent so no merge needed.

Q18. How would you test offline sync?

Mock NetInfo to toggle online/offline, assert queue behavior and Realm state transitions.

Q19. Why GraphQL/AppSync instead of plain REST?

Typed schema, fewer round trips, built-in offline conflict handling features in AppSync, single endpoint for multiple resources.

Part 6: Resume / CV Bullet Points

Q20. What resume bullets can I use for this project?

- Built and maintained a schema-driven dynamic form rendering engine in React Native, supporting conditional field/page/section visibility, dynamic validation, and required/readonly logic driven entirely by JSON configuration.
- Implemented offline-first data persistence using Realm, enabling field users to create, edit, and submit forms without network connectivity, with automatic background sync via an outbox/queue pattern.
- Designed an attachment and annotation module (photo markup + PDF annotation) using `react-native-skia`, supporting freehand drawing, undo/redo history, and serialized vector storage for performant rendering on low-end devices.

- Integrated GraphQL (Apollo Client) with AWS AppSync and a REST fallback layer for form metadata, submissions, and file uploads, with retry and compression logic to handle unreliable network conditions.
- Wrote unit tests (Jest + React Native Testing Library) for core business logic - conditional logic engine, annotation geometry/serialization, sync services - using manual mocks to isolate native modules (Realm, camera, PDF, Skia).
- Diagnosed and fixed [a specific bug you recall] in the sync/annotation flow, improving reliability of form submissions in low-connectivity environments.

One more prep tip: For the bracketed [pick ...] and [a specific bug you recall] spots - fill those in with something you actually did, since a good interviewer will ask you to go deep on whatever you claim, and generic answers fall apart under follow-up questions.